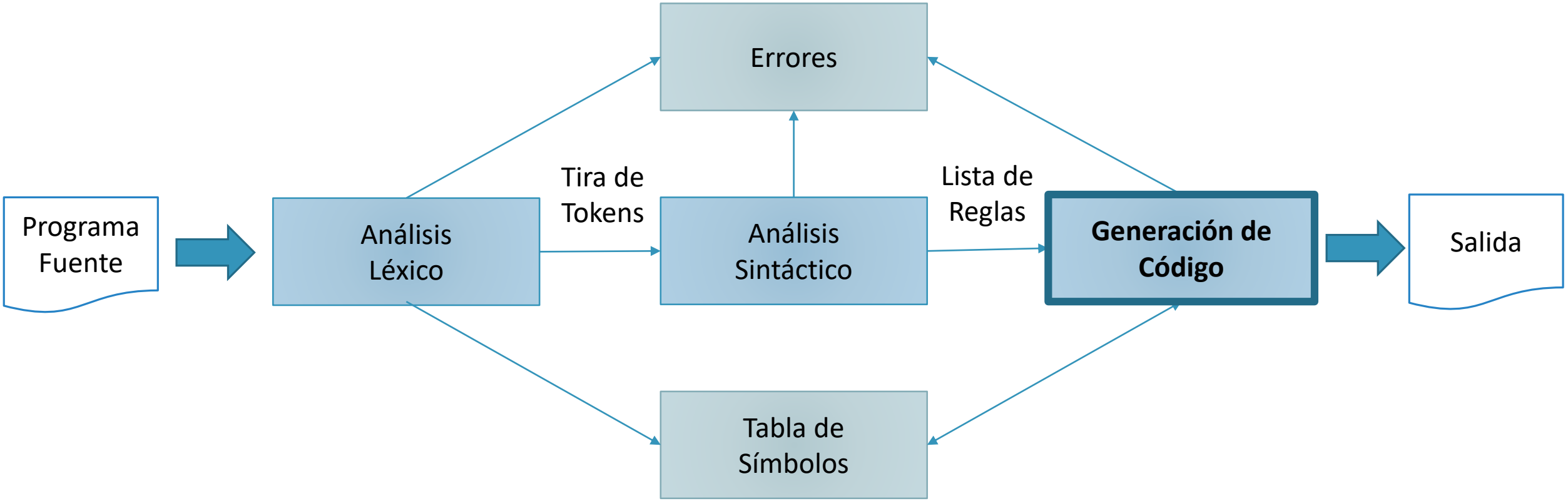
Diseño de Compiladores I

TRABAJO PRACTICO 3

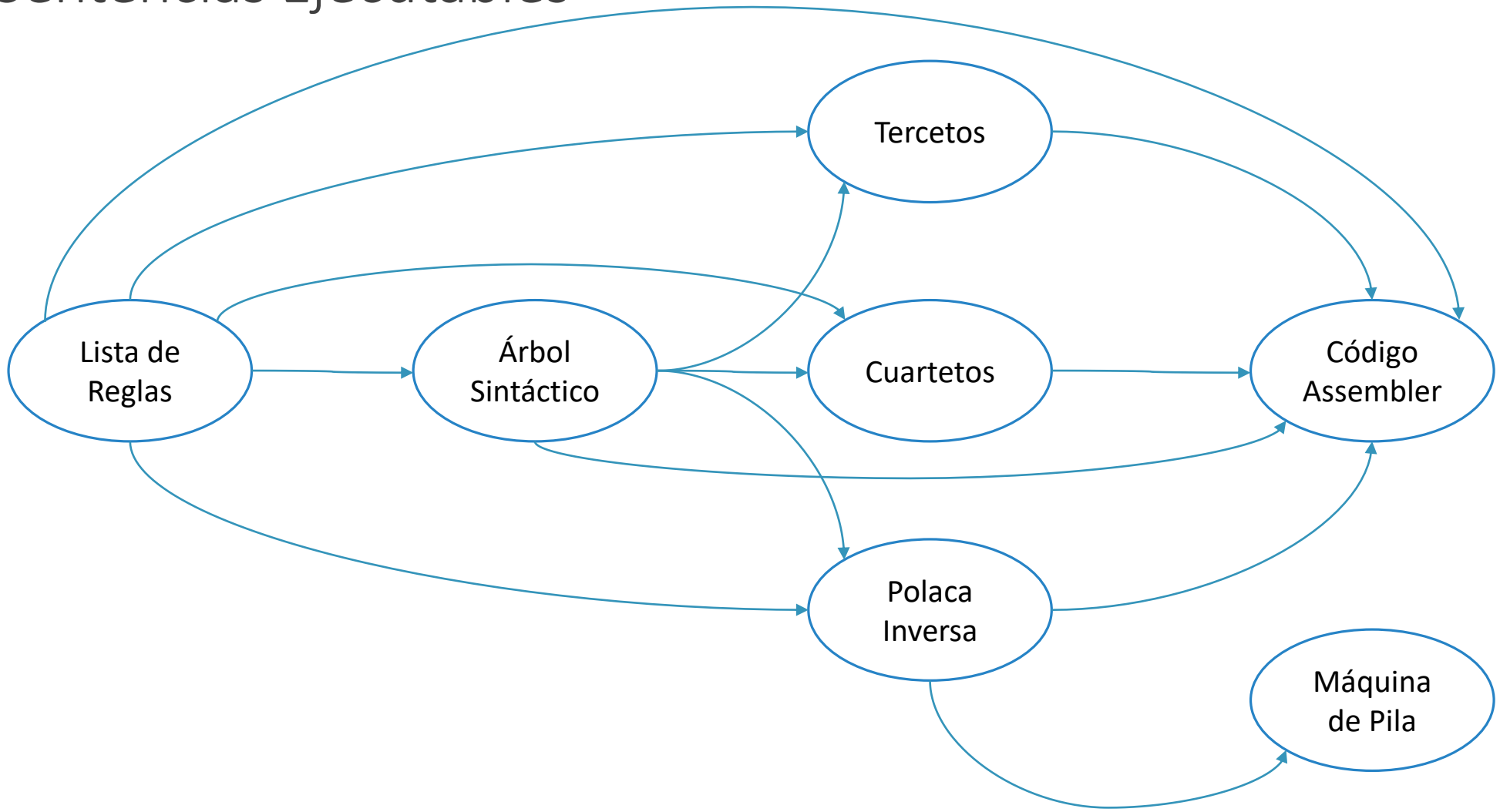
GENERACIÓN DE REPRESENTACIÓN INTERMEDIA

Fases de la Compilación



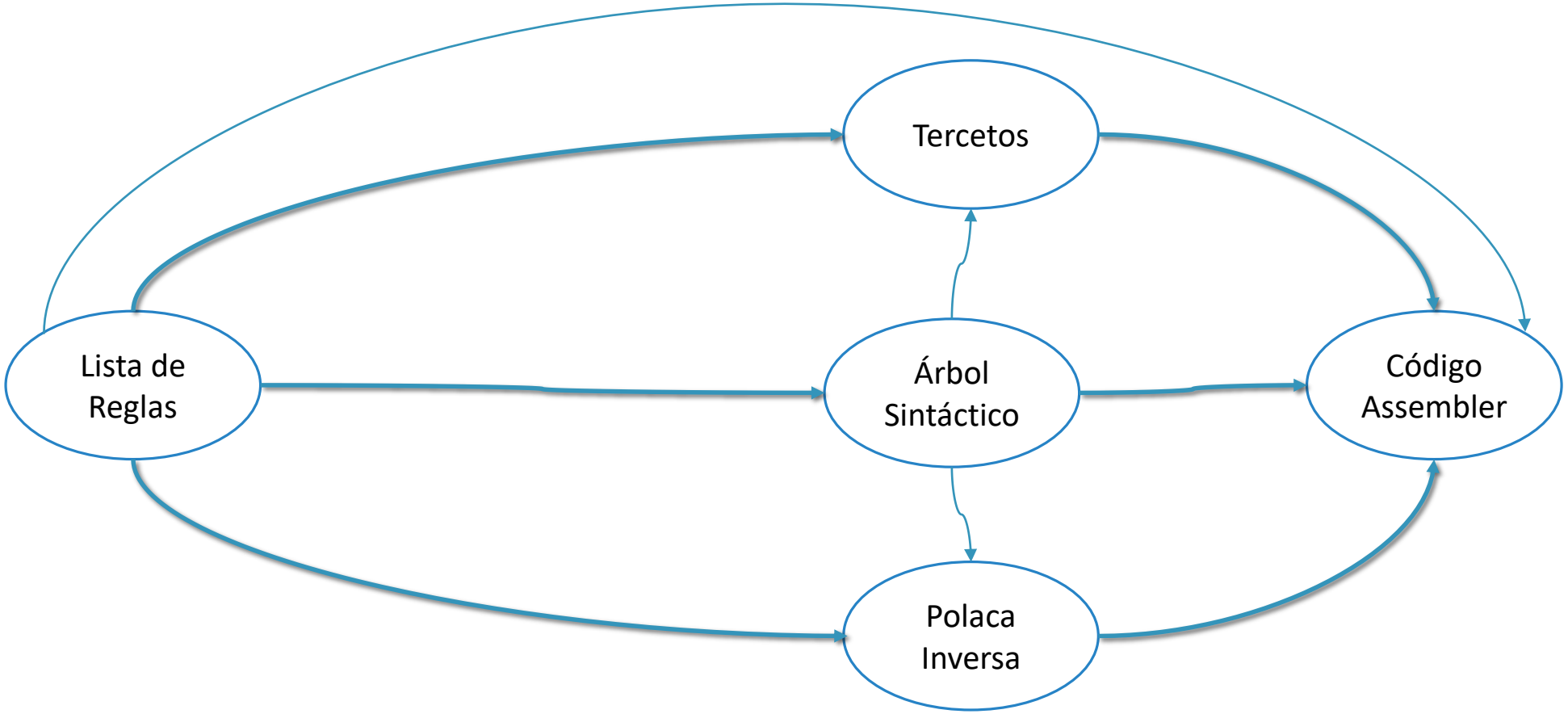
Grafo de Generación de Código

Sentencias Ejecutables



Grafo de Generación de Código

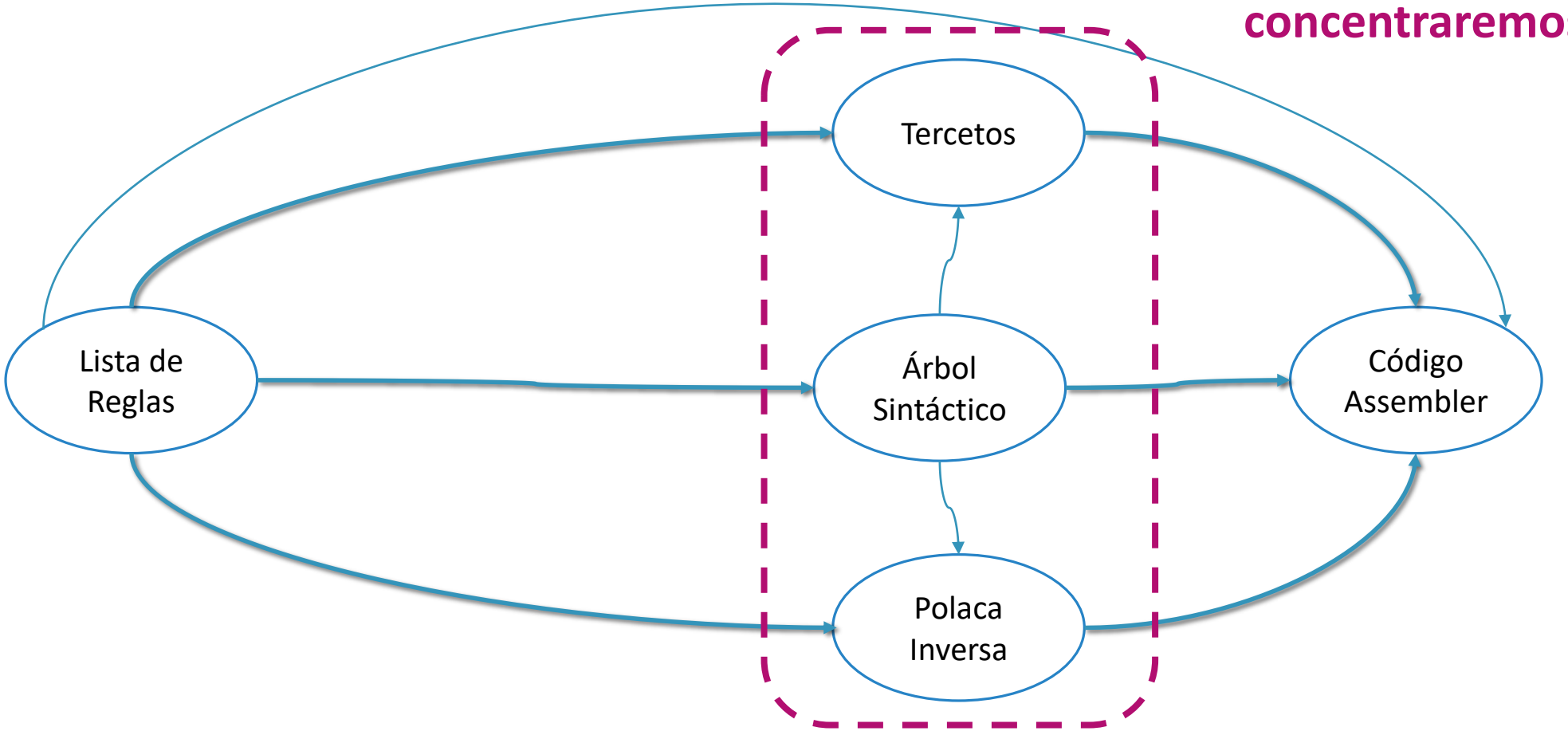
Sentencias Ejecutables



Grafo de Generación de Código

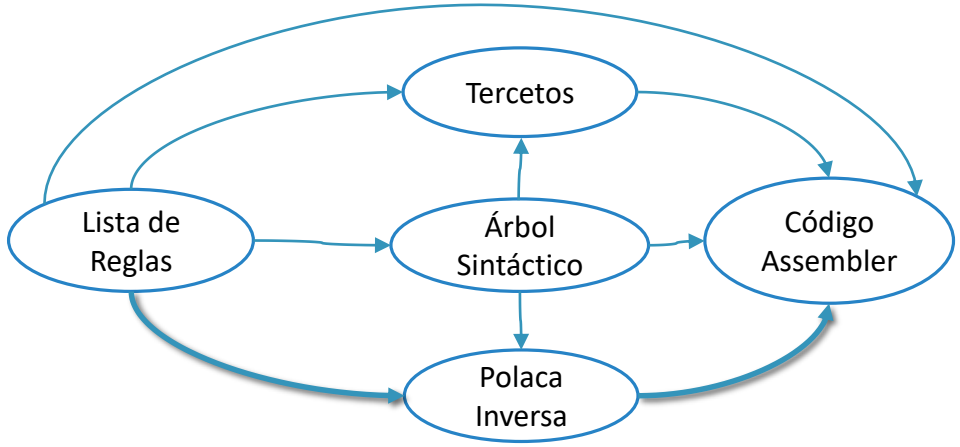
Sentencias Ejecutables

En este práctico, nos concentraremos en esto!



Representación Intermedia

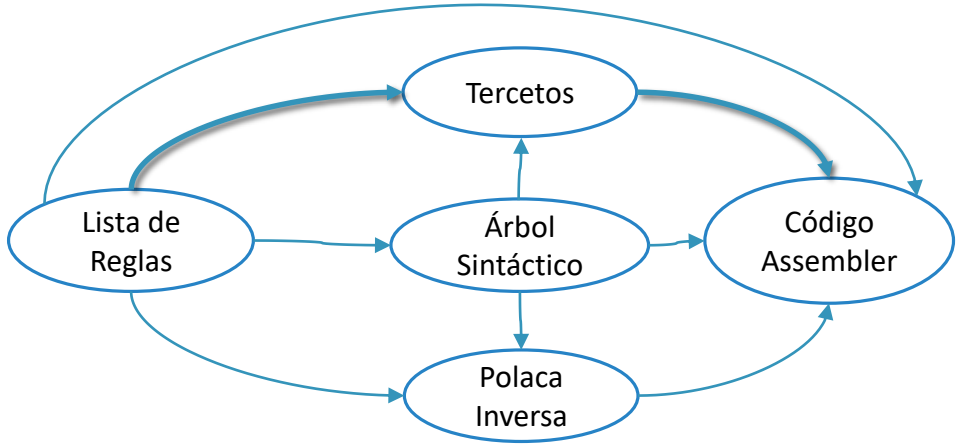
Polaca Inversa



- Operadores Binarios:
  - op1 operador op2
  - op1 op2 operador
- Operadores Unarios:
  - operador op1
  - op1 operador
- Ventajas:
  - Desarrollo rápido.
  - Compilador relativamente rápido.
  - Ejecutable relativamente bueno.

Representación Intermedia

Tercetos



- Representación constituida por códigos de 3 direcciones:

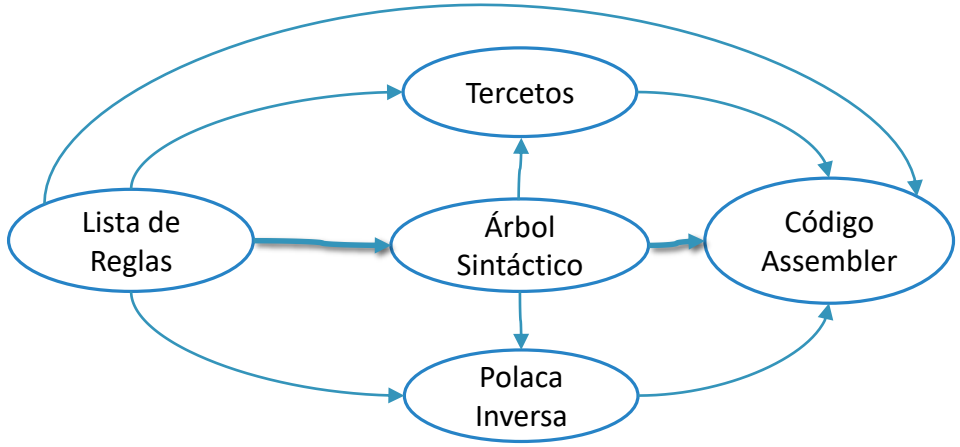
  (operador, op1, op2)

- En el caso de operadores unarios:

  (operador, op1, -)

Representación Intermedia

Árbol Sintáctico



- Es una representación comprimida del árbol de parsing.

- Permite el pasaje a cualquiera de las otras representaciones:

  - Inorden → Código original
  - Postorden → Polaca Inversa
  - Preorden → Tercetos

Sentencias de Asignación

Gramática Reducida

<ASIGNACION> → ID := <EXPRESION>

<EXPRESION> → <EXPRESION> + <TERMINO>

<EXPRESION> → <TERMINO>
<TERMINO> → <FACTOR>

<FACTOR> → ID
<FACTOR> → CONSTANTE

x := y + z

Sentencias de Asignación

Gramática Reducida

```
x := y + z
```

```
<ASIGNACION> → ID := <EXPRESION>
```



Agregar el ID al cuál se le realizó la asignación

```
<EXPRESION> → <EXPRESION> + <TERMINO>
```



Agregar la operación que se realizó

```
<EXPRESION> → <TERMINO>
<TERMINO> → <FACTOR>
```



Agregar el ID o la constante que intervino en la asignación u operación *(Referencias a la Tabla de Símbolos)*

```
<FACTOR> → ID
<FACTOR> → CONSTANTE
```

Sentencias de Asignación

Polaca Inversa

```
x := y + z
```

```
<ASIGNACION> → ID := <EXPRESION>
```

```
Agregar( id.ptr ); Agregar( ":=" )
```

```
<EXPRESION> → <EXPRESION> + <TERMINO>
```

```
Agregar( "+" )
```

```
<EXPRESION> → <TERMINO>
<TERMINO> → <FACTOR>
```

```
<FACTOR> → ID
<FACTOR> → CONSTANTE
```

```
Agregar( id.ptr )
Agregar( cte.ptr )
```

Sentencias de Asignación

Polaca Inversa

x := y + z

<ASIGNACION> → ID := <EXPRESION>

Agregar(id.ptr); Agregar(":=")

<EXPRESION> → <EXPRESION> + <TERMINO>

Agregar("+")

<EXPRESION> → <TERMINO>

<TERMINO> → <FACTOR>

<FACTOR> → ID

<FACTOR> → CONSTANTE

Agregar(id.ptr)

Agregar(cte.ptr)

y	z	+	x	:=

Sentencias de Asignación

Tercetos

```
x := y + z
```

```
<ASIGNACION> → ID := <EXPRESION>
```

A.ptr = crear_terceto(“:=”,id.ptr,E.ptr)

```
<EXPRESION> → <EXPRESION> + <TERMINO>
```

E.ptr = crear_terceto(“+”,E.ptr,T.ptr)

```
<EXPRESION> → <TERMINO>
```

E.ptr = T.ptr

```
<TERMINO> → <FACTOR>
```

T.ptr = F.ptr

```
<FACTOR> → ID
```

F.ptr = id.ptr

```
<FACTOR> → CONSTANTE
```

F.ptr = cte.ptr

Sentencias de Asignación

Tercetos

```
x := y + z
```

<ASIGNACION> → ID := <EXPRESION>

A.ptr = crear_terceto(“:=”,id.ptr,E.ptr)

<EXPRESION> → <EXPRESION> + <TERMINO>

E.ptr = crear_terceto(“+”,E.ptr,T.ptr)

<EXPRESION> → <TERMINO>

E.ptr = T.ptr

<TERMINO> → <FACTOR>

T.ptr = F.ptr

<FACTOR> → ID

F.ptr = id.ptr

<FACTOR> → CONSTANTE

F.ptr = cte.ptr

1. (+ , y , z)
2. (:= ,x , [1])

Sentencias de Asignación

Árbol Sintáctico

x := y + z

<ASIGNACION> → ID := <EXPRESION>

A.ptr = crear_nodo(“:=”,crear_hoja(id.ptr),E.ptr)

<EXPRESION> → <EXPRESION> + <TERMINO>

E.ptr = crear_nodo(“+”,E.ptr,T.ptr)

<EXPRESION> → <TERMINO>

E.ptr = T.ptr

<TERMINO> → <FACTOR>

T.ptr = F.ptr

<FACTOR> → ID

F.ptr = crear_hoja(id.ptr)

<FACTOR> → CONSTANTE

F.ptr = crear_hoja(cte.ptr)

Sentencias de Asignación

Árbol Sintáctico

x := y + z

<ASIGNACION> → ID := <EXPRESION>

A.ptr = crear_nodo(“:=”,crear_hoja(id.ptr),E.ptr)

<EXPRESION> → <EXPRESION> + <TERMINO>

E.ptr = crear_nodo(“+”,E.ptr,T.ptr)

<EXPRESION> → <TERMINO>

E.ptr = T.ptr

<TERMINO> → <FACTOR>

T.ptr = F.ptr

<FACTOR> → ID

F.ptr = crear_hoja(id.ptr)

<FACTOR> → CONSTANTE

F.ptr = crear_hoja(cte.ptr)

Sentencias de Asignación

Árbol Sintáctico

x := y + z

<ASIGNACION> → ID := <EXPRESION>

A.ptr = crear_nodo(“:=”,crear_hoja(id.ptr),E.ptr)

<EXPRESION> → <EXPRESION> + <TERMINO>

E.ptr = crear_nodo(“+”,E.ptr,T.ptr)

<EXPRESION> → <TERMINO>

E.ptr = T.ptr

<TERMINO> → <FACTOR>

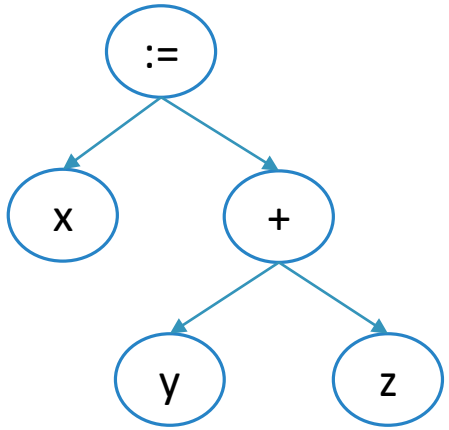
T.ptr = F.ptr

<FACTOR> → ID

F.ptr = crear_hoja(id.ptr)

<FACTOR> → CONSTANTE

F.ptr = crear_hoja(cte.ptr)



Asignación– Comparación Representaciones

REGLAS DE LA GRAMÁTICA	POLACA INVERSA	TERCETOS	ÁRBOL SINTÁCTICO
<ASIGNACION> → ID := <EXPRESION>	Agregar(id.ptr); Agregar(":=")	A.ptr = crear_terceto(":=",id.ptr,E.ptr)	A.ptr = crear_nodo(":=",crear_hoja(id.ptr),E.ptr)
<EXPRESION> → <EXPRESION> + <TERMINO>	Agregar("+")	E.ptr = crear_terceto("+",E.ptr,T.ptr)	E.ptr = crear_nodo("+",E.ptr,T.ptr)
<EXPRESION> → <TERMINO>		E.ptr = T.ptr	E.ptr = T.ptr
<TERMINO> → <FACTOR>		T.ptr = F.ptr	T.ptr = F.ptr
<FACTOR> → ID	Agregar(id.ptr)	F.ptr = id.ptr	F.ptr = crear_hoja(id.ptr)
<FACTOR> → CONSTANTE	Agregar(cte.ptr)	F.ptr = cte.ptr	F.ptr = crear_hoja(cte.ptr)

Sentencias de Control - Ejemplos

Selección

```
if (a > 0) then
    k := k*5;
else
    j := j-3;
```

<SELECCION> →

IF <CONDICION> THEN <BLOQUE> ELSE <BLOQUE>

Iteración

```
while (a > 0)
{
   k := k*5;
   j := j-3;
}
```

<ITERACION> →

WHILE <CONDICION> <BLOQUE>

Sentencias de Control

Gramática Reducida

<SELECCION> → IF <CONDICION> THEN <BLOQUE> ELSE <BLOQUE>

<ITERACION> → WHILE <CONDICION> <BLOQUE>

<CONDICION> → ‘(’ <EXPRESION_LOGICA> ‘)’

<EXPRESION_LOGICA> → <EXPRESION> <SIGNO> <EXPRESION>
<SIGNO> → < | > | <= | >= | !=

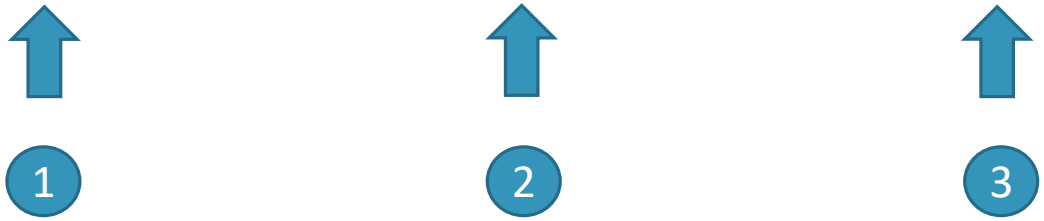
<BLOQUE> := <SENTENCIA> | ‘{’ <SENTENCIAS> ‘}’

<SENTENCIAS> → <SENTENCIAS> <SENTENCIA> | <SENTENCIA>
<SENTENCIA> → <SELECCION> | <ITERACION> | <ASIGNACION> ‘;’

<EXPRESION> → <EXPRESION> + <TERMINO> | <EXPRESION> - <TERMINO> | <TERMINO>
<TERMINO> → <TERMINO> * <FACTOR> | <TERMINO> / <FACTOR> | <FACTOR>
<FACTOR> → <ID> | <DIGITO>

Sentencias de Control - Selección

<SELECCION> → IF <CONDICION> THEN <BLOQUE> ELSE <BLOQUE>



1	TERMINA LA EVALUACIÓN DE LA CONDICIÓN. • Debe generarse el salto por falso (BF). • Todavía no se conoce la dirección a donde se va a saltar. Se tiene una dirección incompleta.
2	FIN DEL BLOQUE THEN. • Debe generarse el salto incondicional (BI) al final de la selección. • Se está en posición de conocer la dirección del BF.
3	FIN DEL BLOQUE ELSE. • Se terminó de reconocer la sentencia de selección. • Se está en posición de conocer la dirección del BI.

Sentencias de Control - Selección

<SELECCION> → IF **<CONDICION>** THEN **<BLOQUE>** ELSE **<BLOQUE>**



<SELECCION> → IF **<CONDICION_IF>** THEN **<BLOQUE_THEN>** ELSE **<BLOQUE_ELSE>**

```
<SELECCION> := IF <CONDICION_IF> THEN <BLOQUE_THEN> ELSE <BLOQUE_ELSE>

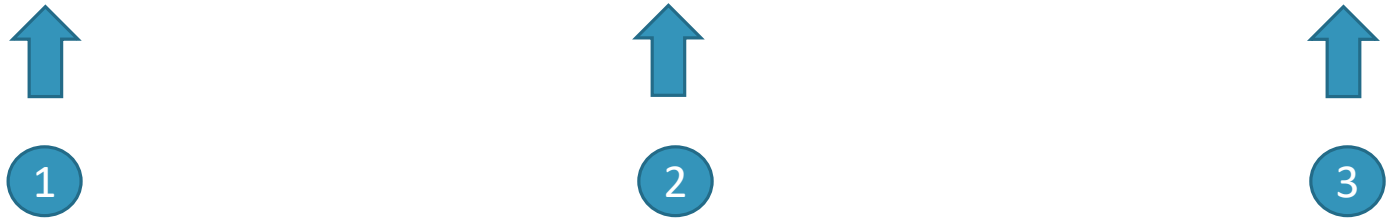
<CONDICION_IF> := <CONDICION>

<BLOQUE_THEN> := <BLOQUE>

<BLOQUE_ELSE> := <BLOQUE>
```

Selección – Polaca Inversa

<SELECCION> → IF <CONDICION_IF> THEN <BLOQUE_THEN> ELSE <BLOQUE_ELSE>



1	TERMINA LA EVALUACIÓN DE LA CONDICIÓN. (Generar BF) 1. Crear espacio en blanco 2. Apilar la dirección del paso incompleto. 3. Crear el paso del BF.
2	FIN DEL BLOQUE THEN. (Se conoce la dirección destino del BF y necesita un BI) 1. Desapilar la dirección del paso incompleto y completar con *#_paso_actual* + 2(*) 2. Crear espacio en blanco. 3. Apilar la dirección del paso incompleto. 4. Crear el paso del BF.
3	FIN DEL BLOQUE ELSE. (Se conoce la dirección destino del BI) 1. Desapilar la dirección del paso incompleto. 2. Completar con *#_paso_actual* + 1.

Selección – Polaca Inversa

Acciones Semánticas

<SELECCION> → IF <CONDICION_IF> THEN <BLOQUE_THEN> ELSE <BLOQUE_ELSE>

1 2 3

	REGLAS DE LA GRAMÁTICA	ACCIONES SEMÁNTICAS
3	<SELECCION> → IF <CONDICION_IF> THEN <BLOQUE_THEN> ELSE <BLOQUE_ELSE>	#_paso_incomp = desapilar_paso(); ***//Desapila dirección incompleta.*** completar_paso(#_paso_incomp, #_paso_actual + 1); ***//Completa el destino de BI.***
1	<CONDICION_IF> → <CONDICION>	#_paso_actual = create_paso(" "); ***//Crea paso incompleto.*** apilar_paso(#_paso_actual); ***//Apila el número del paso incompleto.*** #_paso_actual = create_paso("BF"); ***//Crea el paso BF.***
2	<BLOQUE_THEN> → <BLOQUE>	#_paso_incomp = desapilar_paso(); ***//Desapila dirección incompleta.*** completar(#_paso_incomp , #_paso_actual + 2); ***//Completa el destino de BF.*** #_paso_actual = create_paso(" "); ***//Crea paso incompleto.*** apilar_paso(#_paso_actual); ***//Apila el número del paso incompleto.*** #_paso_actual = generar_paso("BI"); ***//Se crea el paso BI.***
	<BLOQUE_ELSE> → <BLOQUE>	-

Selección – Polaca Inversa

Ejemplo

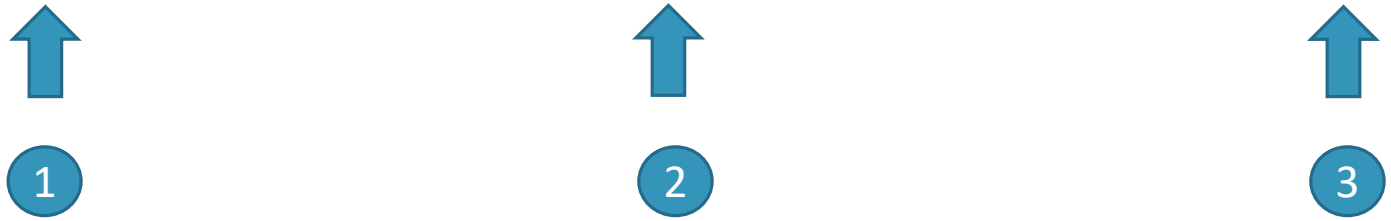
```
if (a > 0) then
   k := k*5;
else
   j := j-3;
```

a	0	>	13	BF	5	k	*	k	:=
1	2	3	4	5	6	7	8	9	10

18	BI	j	3	-	j	:=	
11	12	13	14	15	16	17	18

Selección – Tercetos

<SELECCION> → IF <CONDICION_IF> THEN <BLOQUE_THEN> ELSE <BLOQUE_ELSE>



1	TERMINA LA EVALUACIÓN DE LA CONDICIÓN. (Generar BF) 1. Crear el tercero incompleto correspondiente al BF. 2. Apilar el número de terceto incompleto.
2	FIN DEL BLOQUE THEN. (Se conoce la dirección destino del BF y necesita un BI) 1. Desapilar el número de terceto incompleto (BF) y completar con DIR_ACTUAL+2(*) 2. Crear terceto incompleto (BI). 3. Apilar el número de terceto incompleto.
3	FIN DEL BLOQUE ELSE. (Se conoce la dirección destino del BI) 1. Desapilar el número del terceto incompleto (BI). 2. Completar con DIR_ACTUAL+1.

Selección – Tercetos

Acciones Semánticas

<SELECCION> → IF <CONDICION_IF> THEN <BLOQUE_THEN> ELSE <BLOQUE_ELSE>

1 2 3

	REGLAS DE LA GRAMÁTICA	ACCIONES SEMÁNTICAS
3	<SELECCION> → IF <CONDICION_IF> THEN <BLOQUE_THEN> ELSE <BLOQUE_ELSE>	#_terc_incomp = desapilar_terceto(); completar_terceto(#_terc_incomp, #_terc_actual + 1); ***//Completa el terceto BI.***
1	<CONDICION_IF> → <CONDICION>	#_terc_actual = crear_terceto("BF",Cond.prt, **[?]**); ***//Crea un terceto incompleto.*** *apilar_terceto(*#_terc_actual*);* ***//Apila el número del terceto incompleto.***
2	<BLOQUE_THEN> → <BLOQUE>	#_terc_incomp = desapilar_terceto(); completar_terceto(#_terc_incomp, #_terc_actual + 2); ***//Completa el terceto BF.*** #_terc_actual = crear_terceto("BI", **[?]**, -); ***//Crea un terceto incompleto.*** apilar_terceto(#_terc_actual); ***//Apila el número del terceto incompleto.***
	<BLOQUE_ELSE> → <BLOQUE>	-

Selección – Tercetos

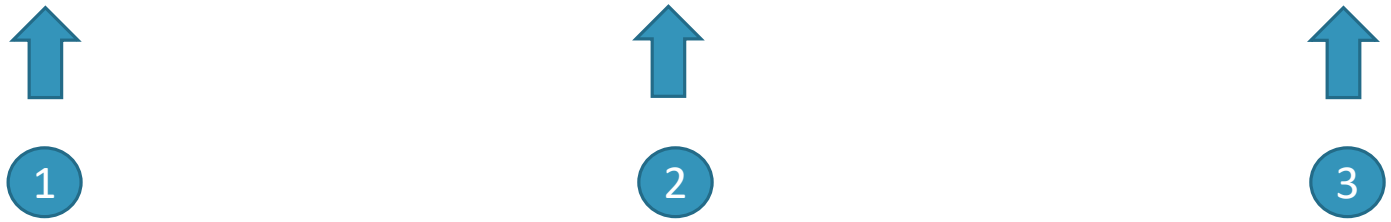
Ejemplo

```
if (a > 0) then
   k := k*5;
else
   j := j-3;
```

```
1.( < , a , 0 )
2.( BF , [1], 6 )
3.( * , k , 5 )
4.( := , k , [3] )
5.( BI , 8 , -- )
6.( - , j , 3 )
7.( := , j , [6] )
```

Selección – Árbol Sintáctico

<SELECCION> → IF <CONDICION_IF> THEN <BLOQUE_THEN> ELSE <BLOQUE_ELSE>



1	TERMINA LA EVALUACIÓN DE LA CONDICIÓN. Crear el nodo correspondiente a la condición evaluada.
2	FIN DEL BLOQUE THEN. Crear el nodo correspondiente al bloque THEN. (Apuntando a bloque de sentencias correspondiente).
3	FIN DEL BLOQUE ELSE. Crear el nodo correspondiente al bloque ELSE (Apuntando a bloque de sentencias correspondiente). 1. Crear el nodo correspondiente al cuerpo del IF (Apuntando a los nodos del bloque THEN y ELSE). 2. Crear el nodo correspondiente al IF (Apuntando al nodo de la condición y del cuerpo).

Selección – Árbol Sintáctico

Acciones Semánticas

<SELECCION> → IF <CONDICION_IF> THEN <BLOQUE_THEN> ELSE <BLOQUE_ELSE>

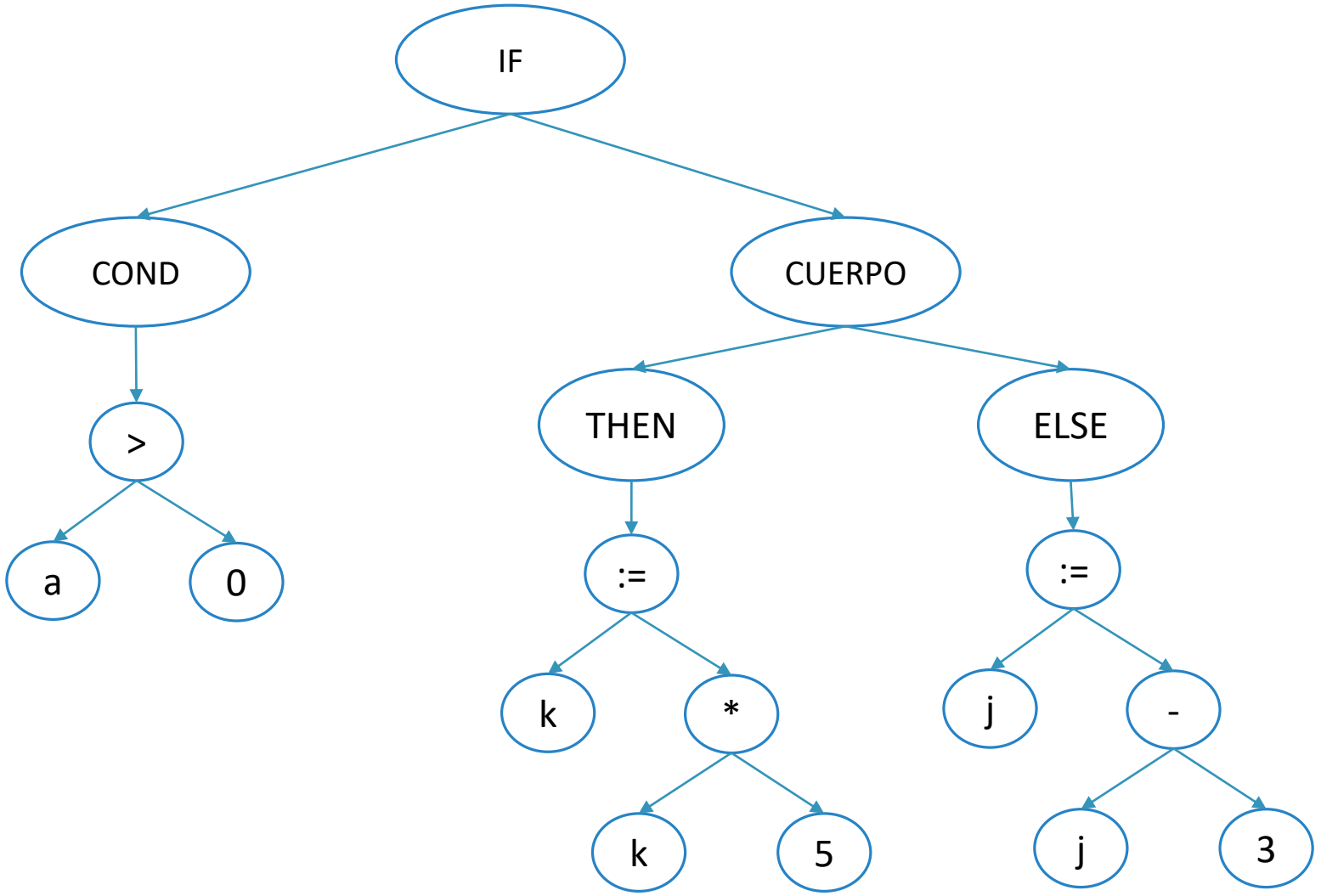
1 2 3

	REGLAS DE LA GRAMÁTICA	ACCIONES SEMÁNTICAS
3	<SELECCION> → IF <CONDICION_IF> THEN <BLOQUE_THEN> ELSE <BLOQUE_ELSE>	Cuerpo_IF.ptr = crear_nodo("cuerpo", Bloque_then.ptr, Bloque_else.ptr); IF.ptr = crear_nodo("IF", Cond_IF.ptr, Cuerpo_IF.ptr);
1	<CONDICION_IF> → <CONDICION>	Cond_IF.ptr = crear_nodo("cond", Cond.ptr, null);
2	<BLOQUE_THEN> → <BLOQUE>	Bloque_Then.ptr = crear_nodo ("then", Bloque.ptr, null);
	<BLOQUE_ELSE> → <BLOQUE>	Bloque_Else.ptr = crear_nodo ("else", Bloque.ptr, null);

Selección – Árbol Sintáctico

Ejemplo

```
if (a > 0) then
   k := k*5;
else
   j := j-3;
```

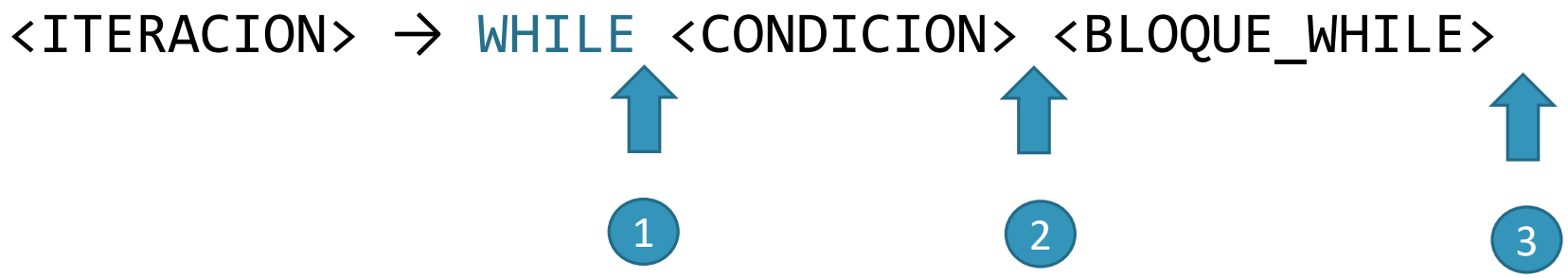


Selección – Comparación Representaciones

REGLAS DE LA GRAMÁTICA	POLACA INVERSA	TERCETOS	ÁRBOL SINTÁCTICO
<SELECCION> → IF <CONDICION_IF> THEN <BLOQUE_THEN> ELSE <BLOQUE_ELSE>	#_paso_incomp = desapilar_paso(); completar_paso(#_paso_incomp, #_paso_actual + 1);	#_terc_incomp = desapilar_terceto(); completar_terceto(#_terc_incomp, #_terc_actual + 1);	Cuerpo_IF.ptr = crear_nodo("cuerpo", Bloque_then.ptr, Bloque_else.ptr); IF.ptr = crear_nodo("IF", Cond_IF.ptr, Cuerpo_IF.ptr);
<CONDICION_IF> → <CONDICION>	#_paso_actual = create_paso(" "); apilar_paso(#_paso_actual); #_paso_actual = create_paso("BF");	#_terc_actual = crear_terceto("BF",Cond.prt, **[?]**); apilar_terceto(#_terc_actual);	Cond_IF.ptr = crear_nodo("cond", Cond.ptr, null);
<BLOQUE_THEN> → <BLOQUE>	#_paso_incomp = desapilar_paso(); completar(#_paso_incomp , #_paso_actual + 2); #_paso_actual = create_paso(" "); apilar_paso(#_paso_actual); #_paso_actual = generar_paso("BI");	#_terc_incomp = desapilar_terceto(); completar_terceto(#_terc_incomp, #_terc_actual + 2); #_terc_actual = crear_terceto("BI", **[?]**, -); apilar_terceto(#_terc_actual);	Bloque_Then.ptr = crear_nodo ("then", Bloque.ptr, null);
<BLOQUE_ELSE> → <BLOQUE>	-	-	Bloque_Else.ptr = crear_nodo ("else", Bloque.ptr, null);

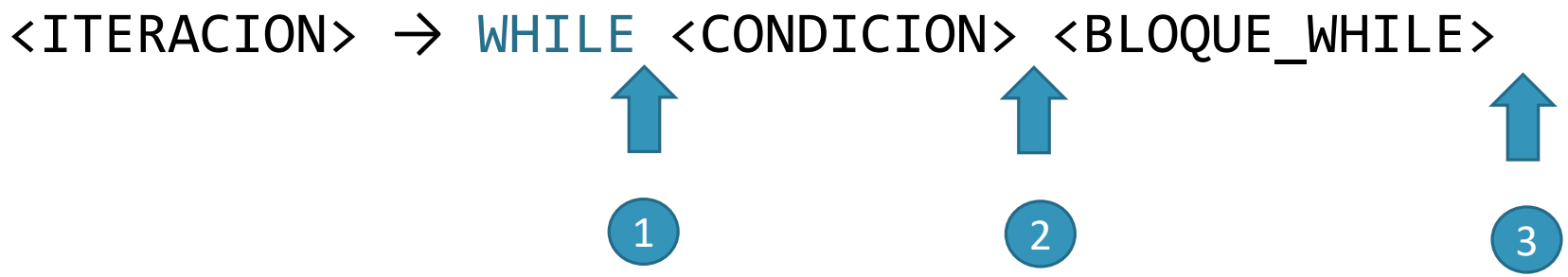
Iteración

¿Qué deberíamos tener en cuenta?



Iteración

¿Qué deberíamos tener en cuenta?

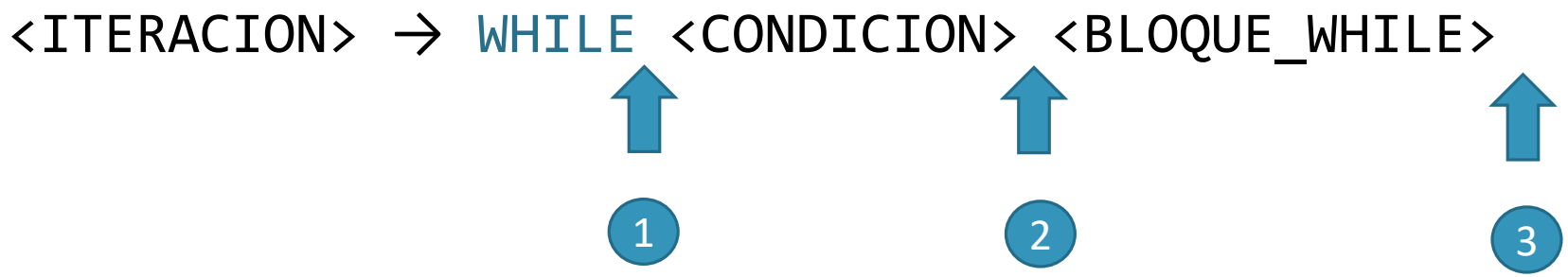


Polaca Inversa

1	Apilar el número de paso donde comienza la condición.
2	Generar los pasos de la BF incompleta. Apilar el número de paso incompleto.
3	Desapilar. Completar el paso incompleto con el destino de la BF. Desapilar paso de inicio. Generar los pasos de la BI.

Iteración

¿Qué deberíamos tener en cuenta?



Terceto

1	Apilar el número de terceto donde comienza la condición.
2	Crear terceto incompleto para la BF. Apilar el número de terceto incompleto.
3	Desapilar. Completar el terceto incompleto con el destino de la BF. Desapilar número de terceto de inicio. Generar el terceto para la BI.

Iteración

¿Qué deberíamos tener en cuenta?

<ITERACION> → WHILE <CONDICION> <BLOQUE_WHILE>



Árbol Sintáctico

1	Crear el nodo de control “COND”.
2	Crear el nodo de control “BLOQUE_WHILE”.
3	Crear_nodo(“WHILE”,Condicion.ptr,Bloque_While.ptr)

¿Qué tienen que entregar?

- **Código intermedio**, de alguna forma que permita **visualizarlo claramente**.
  - Tercetos y Polaca Inversa → Mostrar la dirección de cada elemento.
  - Árbol Sintáctico → Forma de presentación que permita visualizar la estructura del árbol.

Ejemplos:

Tercetos	Polaca Inversa	Árbol Sintáctico
20 (+ , a , b) 21 (/ , [20] , 5) 22 (= , z , [21])	10 < 11 a 12 b 13 18 14 BF 15 c 16 10 17 = 18 ...	Raíz Hijo izquierdo Hijo izquierdo Hijo derecho Hijo derecho Hijo izquierdo ... Hijo derecho

¿Qué tienen que entregar?

- Código intermedio debe contener referencias a la Tabla de Símbolos, en la visualización del código se deberán **mostrar los nombres de los símbolos** correspondientes.

- Los **errores generados en cada una de las etapas** (Análisis Léxico, Sintáctico y Generación de Código Intermedio) se deberán mostrar **todos juntos**:
  - Descripción de cada error.
  - Línea.

- **NO** se **deberán mostrar las estructuras sintácticas ni los tokens** (salidas TP 1 y 2)
  - A menos que se lo solicite! ;)

- **Contenido de la Tabla de Símbolos.**

Diseño de Compiladores I

TRABAJO PRACTICO 3

GENERACIÓN DE REPRESENTACIÓN INTERMEDIA